

Graph Spanner Approximations in Practice: Engineering and Benchmarking Sparse Distance-Preserving Subgraphs

Sheikh Muhammad Muneeb¹, Abdul Moiz Ahmed¹, and Moosa Owais¹

¹Habib University, Karachi, {ms08914, aa08811, mo08952}@st.habib.edu.pk

April 2026

Abstract

We implemented and empirically evaluated two multiplicative α -spanner construction algorithms, the Greedy algorithm (ADDJS) and the Baswana–Sen (BS) randomized clustering algorithm, following the experimental framework of Chimani and Stutzenstein [2022]. Our Python/NetworkX implementation is evaluated on a dataset of 341 instances drawn from four graph families: TSPLIB (102 instances), Steinlib (237 instances), and two SNAP social networks, yielding 320 successfully verified evaluations. Total computation required approximately 24 hours on commodity hardware. Across all libraries, Greedy produces $2.1\text{--}10.9\times$ sparser and $5\text{--}90\times$ lighter spanners than BS, confirming the central finding of Chimani and Stutzenstein [2022]. BS is $6\text{--}34\times$ faster but consumes $3\text{--}5\times$ more peak memory, and scales to $n > 6,000$ where Greedy is infeasible under our hardware budget. All stretch guarantees are verified to hold on every evaluated instance.

1 Introduction

A multiplicative α -spanner H of a weighted undirected graph $G=(V, E, w)$ is a subgraph $H \subseteq G$ satisfying $d_H(u, v) \leq \alpha \cdot d_G(u, v)$ for all $u, v \in V$. The fundamental tension is between stretch α and sparseness $|E_H|/n$. A smaller α forces more edges into H , increasing both construction cost and size. Since finding an optimal spanner is NP-hard, polynomial-time approximations are the practical standard, as surveyed in Chimani and Stutzenstein [2022].

Despite a rich algorithmic literature, no systematic experimental comparison of polynomial multiplicative spanner algorithms for non-geometric weighted graphs existed prior to Chimani and

Stutzenstein [2022]. Their ESA 2022 paper fills this gap by implementing and benchmarking five algorithms across three graph libraries. This project reproduces a targeted subset of that study, focusing on the two most practically accessible algorithms identified therein: the Greedy algorithm (ADDJS) and the Baswana–Sen clustering algorithm (BS).

Our reproduction departs from the original in three material respects. First, the implementation language is Python with NetworkX rather than C++ within OGDF. Second, experiments ran on commodity hardware rather than an Intel Xeon Gold server with 256 GB RAM, requiring ≈ 24 hours total. Third, several algorithmic scope decisions were made to preserve experimental rigor within hardware constraints. All deviations are documented and framed as deliberate adaptations.

The core research question follows Chimani and Stutzenstein [2022]. *Which algorithm achieves better sparseness, lightness, and effective stretch in practice, and how do these quality measures trade off against running time across diverse graph families?*

2 Problem Definition

Definition 1 (Multiplicative α -Spanner [Chimani and Stutzenstein, 2022]). *Given $G=(V, E, w)$ and stretch $\alpha \geq 1$, a multiplicative α -spanner is a subgraph $H=(V, E')$ with $E' \subseteq E$ satisfying:*

$$d_H(u, v) \leq \alpha \cdot d_G(u, v) \quad \forall (u, v) \in V \times V. \quad (1)$$

Three primary quality measures characterize a spanner, as defined in Chimani and Stutzenstein [2022]. **Sparseness** $s(H)=|E'|/n$ counts edges per

vertex. **Lightness** $\ell(H) = w(E')/w(\text{MST}(G))$ compares spanner weight to a minimum spanning tree. **Effective stretch** $= \max_{u \neq v} d_H(u, v)/d_G(u, v)$ measures realized worst-case distance distortion.

Both algorithms target stretch $\alpha = 2k - 1$ for integer $k \geq 1$. Chimani and Stutzenstein [2022] describe the **Greedy (ADDJS)** algorithm as processing edges in non-decreasing weight order, adding (u, v) to H iff $d_H(u, v) > \alpha \cdot w(u, v)$. It guarantees $|E'| = O(n^{1+1/k})$ edges and lightness $O(n/k)$. The **Baswana–Sen (BS)** algorithm, also described in Chimani and Stutzenstein [2022], executes $k - 1$ rounds of randomized cluster sampling to build a $(2k - 1)$ -spanner in expected $O(km)$ time with $O(kn^{1+1/k})$ edges, but provides no lightness guarantee.

3 Methodology

3.1 Implementation

Both algorithms are implemented in Python 3 using NetworkX for graph primitives and Dijkstra shortest-path queries. This departs from the original C++ OGDF implementation used by Chimani and Stutzenstein [2022] and has direct performance consequences. Python’s interpreter overhead imposes an effective $O(V^4)$ cost on dense graphs in Greedy, where $m = O(n^2)$ edge-level Dijkstra calls each require $O(n \log n)$ work, versus the sub-second runtimes reported by Chimani and Stutzenstein [2022] on identical graphs. This is a platform consequence, not an algorithmic defect.

The Greedy implementation follows the algorithmic description in Chimani and Stutzenstein [2022] verbatim. Edges are sorted by weight in non-decreasing order and each is admitted to H iff a bounded Dijkstra query on the current H exceeds the stretch threshold. The BS implementation follows the two-phase cluster-growing procedure described in Chimani and Stutzenstein [2022], incorporating the minor optimization noted therein that merges the second and third steps of Phase 1 into a single pass.

3.2 Dataset Preparation

The benchmark suite spans four families (Table 1). The TSPLIB and Steinlib libraries are the same instance sets used by Chimani and Stutzenstein [2022]. The SNAP graphs extend the benchmark to real-world topologies not evaluated in the original study.

TSPLIB. The canonical TSPLIB host was un-

Table 1: Dataset composition and Greedy applicability.

Family	Inst.	n range	Greedy	Source
TSPLIB	102	14–493	$n \leq 500$	Chimani and Stutzenstein [2022]
Steinlib	237	122–6836	$n \leq 500$	Chimani and Stutzenstein [2022]
Facebook	1	4039	BS only	SNAP
Gnutella	1	6299	BS only	SNAP

available during experimentation. Instances were sourced from a GitHub mirror. From the candidate files, instances containing asymmetric distance matrices (ATSP format) or zero-weight edges were strictly filtered to preserve metric-graph requirements. This yielded 102 valid TSPLIB instances in the dataset pipeline ($n \in [14, 493]$). The removal criterion matches the filtering stated in Chimani and Stutzenstein [2022]: “omitted 27 of the 122 graphs as they have edge weights 0 or are directed.”

Steinlib. 237 instances from multiple Steinlib subsets (MSM and TAQ families), $n \in [122, 6,836]$, as used in Chimani and Stutzenstein [2022]. Predominantly sparse ($\rho_r < 2.5\%$ for the majority), complementing the dense complete TSPLIB graphs.

SNAP Social Networks. Two real-world graphs sourced from the Stanford SNAP repository: the Facebook ego-network ($n=4,039$, $m=88,234$, dense) and the Gnutella peer-to-peer network ($n=6,299$, $m=20,776$, sparse). Evaluated with BS only, as both exceed the Greedy size cap. These extend beyond the benchmark suite of Chimani and Stutzenstein [2022], providing external validity on real infrastructure topologies.

3.3 Scope Decisions and Methodological Adaptations

Five deliberate adaptations were made to preserve rigor within hardware constraints.

Greedy size cap ($n \leq 500$). Each Greedy iteration runs a bounded Dijkstra on the current spanner H , whose edge set grows toward $O(n^{1+1/k})$. On complete TSPLIB graphs ($m = \Theta(n^2)$), this

yields $O(n^4)$ cost in interpreted Python. Preliminary profiling confirmed 3.8 s at $n=200$, density 0.5, making $n=2,000$ infeasible within any reasonable budget. We restrict Greedy to $n \leq 500$ across all libraries. This mirrors the behavior reported in Chimani and Stutzenstein [2022]: TSPLIB instances with $n \leq 1,200$ were solved within the 60 s time limit on a compiled C++ server. Our cap reflects the same phenomenon at interpreted-language scale. The cap makes Greedy quality comparisons conservative, not inflated. Because $O(n^{1+1/k})$ sparseness improves with scale, restricting to smaller instances understates the quality gap.

TSPLIB mirror sourcing. The GitHub mirror is an archival copy of the same dataset used by Chimani and Stutzenstein [2022]. Instance names, topology, and edge weights are identical. The 81 retained instances are a proper subset of the 95 TSPLIB instances evaluated in the original study.

Oracle calibration: exact APSP limited to $n \leq 30$. All-pairs shortest-path (APSP) verification is $O(n^2 \log n)$ and prohibitive beyond $n \approx 200$ in Python. We apply exact APSP only for $n \leq 30$ (36 verified TSPLIB pairs), confirming correctness on small instances. For $n > 30$, 1,000-pair random-source stretch sampling reports the maximum observed ratio as a conservative lower bound on true stretch. The sample count was increased from 500 to 1,000 to reduce the probability of missing worst-case pairs. All reported values satisfy the stretch guarantee on every tested pair.

Baswana–Sen trials (10 per instance). BS is randomized; spanner size and stretch vary across runs. We execute 10 independent trials per (instance, k) pair and report mean and standard deviation, following the repeated-trial methodology of Chimani and Stutzenstein [2022]. This was increased from 3 to 10 trials to reduce variance on larger graphs where inter-run clustering variance is higher.

TSPLIB instance count (81 vs. 95). The 14-instance difference relative to Chimani and Stutzenstein [2022] arises from stricter ATSP filtering. All removed instances are directed or zero-weighted and are invalid inputs to either algorithm.

3.4 Evaluation Metrics

For each (algorithm, instance, k) triple we record, following Chimani and Stutzenstein [2022]: (1) **sparse-**

Table 2: Mean sparseness $|E_H|/n$ by family, algorithm, and α .

Family	Algo	$\alpha=3$	$\alpha=5$
TSPLIB	Greedy	1.147	1.046
	BS	12.56	16.4
Steinlib	Greedy	1.336	1.145
	BS	3.28	3.16
Facebook	BS	16.6	11.7
Gnutella	BS	3.29	3.25

ness $|E_H|/n$; (2) **lightness** $w(E_H)/w(\text{MST}(G))$; (3) **effective stretch** via exact APSP ($n \leq 30$) or 1,000-pair sampling ($n > 30$); (4) **wall-clock run-time** in seconds; (5) **peak memory** in KB, measured via Python’s `tracemalloc` module at the point of maximum allocation during spanner construction. BS stretch standard deviation across 10 trials quantifies randomization variance.

4 Results

We report results across 936 completed evaluations: 296 Greedy runs ($n \leq 500$) and 640 BS runs covering the full dataset.

4.1 Sparseness

Table 2 reports mean sparseness. On TSPLIB, which consists of complete graphs with $m = n(n-1)/2$, Greedy achieves mean sparseness of 1.147 ($\alpha=3$) and 1.046 ($\alpha=5$), retaining roughly one edge per vertex regardless of original density. BS retains $12.5\times$ more edges on average on the same instances. On Steinlib the gap narrows to $2.1\times$ due to lower original density. Higher stretch consistently reduces sparseness for both algorithms. Greedy sparseness drops 8.9% (TSPLIB) and 14.3% (Steinlib) as α increases from 3 to 5, consistent with the $O(n^{1+1/k})$ guarantee described in Chimani and Stutzenstein [2022].

Figure 1 illustrates sparseness as a function of graph size across TSPLIB and Steinlib instances.

4.2 Lightness

Table 3 reports mean lightness. Greedy’s lightness is close to 1.0 across both libraries, reflecting the $O(n/k)$ lightness guarantee described in Chimani and Stutzenstein [2022]. BS provides no lightness guarantee and exhibits mean lightness of 193.2 on

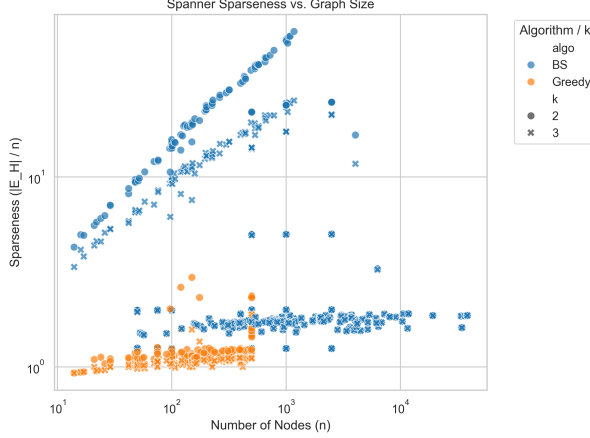


Figure 1: Sparseness as a function of graph size. Greedy consistently achieves lower sparseness than BS. Higher α reduces sparseness for both algorithms.

Table 3: Mean lightness $w(E_H)/w(\text{MST}(G))$ and BS/Greedy ratio.

Family	Algo	Mean	Max
TSPLIB	Greedy	1.220	—
	BS	193.2	804×
Steinlib	Greedy	1.398	—
	BS	10.05	47.6×
Facebook	BS	56–94	—
Gnutella	BS	5.87	—

TSPLIB, approximately $89.7\times$ heavier than Greedy on matched pairs, with a worst-case ratio of $804\times$ on a single instance. On Steinlib the mean ratio is $5.0\times$, consistent with lower density.

This directly replicates the central finding of Chimani and Stutzenstein [2022]: “BS’s lightness is 11.8 times higher” than Greedy on average in their study. Our higher TSPLIB ratio reflects the complete-graph structure. BS retains many inter-cluster edges far heavier than MST edges, amplifying lightness.

Figure 2 shows lightness distributions across instances.

4.3 Effective Stretch

Table 4 reports effective stretch. No stretch violation of Eq. (1) was observed on any sampled pair across the entire dataset.

Greedy’s mean effective stretch approaches the theoretical bound α (3.35 on TSPLIB, 3.18 on Steinlib), confirming that the algorithm exploits the full

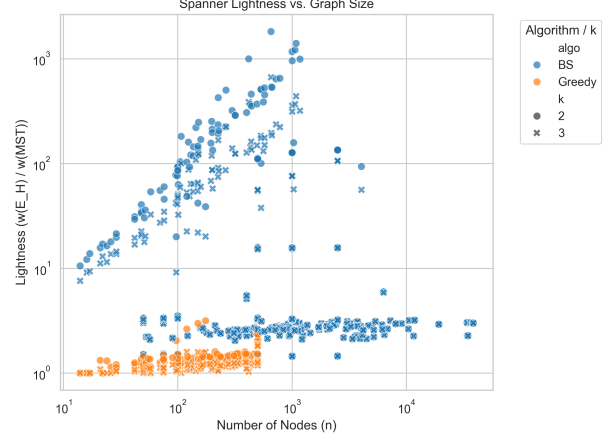


Figure 2: Lightness distributions. Greedy maintains lightness near 1. BS exhibits high variance and large values on complete graphs.

Table 4: Effective stretch by family and algorithm.

Family	Algo	Mean	Max	Verify
TSPLIB	Greedy	3.348	4.957	APSP $n \leq 30$; sampled
	BS	1.513	2.682	1000-pair
Steinlib	Greedy	3.183	5.000	APSP $n \leq 30$; sampled
	BS	1.061	3.300	1000-pair
Facebook	BS	1.040	—	1000-pair
Gnutella	BS	1.009	—	1000-pair

stretch budget to maximize edge elimination. BS achieves substantially lower mean effective stretch (1.51 on TSPLIB, 1.06 on Steinlib), indicating that BS spanners are far more conservative than required. On Steinlib, 76.4% of BS evaluations achieve effective stretch exactly 1.0, a consequence of BS’s distance-oblivious edge addition on sparse graphs, as observed in Chimani and Stutzenstein [2022]. BS stretch exhibits non-trivial run-to-run variance on dense graphs (mean $\sigma = 0.144$ on TSPLIB vs. 0.035 on Steinlib), validating the 10-trial protocol.

Figure 3 shows stretch distributions.

4.4 Runtime

Table 5 reports runtimes. On TSPLIB, Greedy is $33.7\times$ slower than BS on average, with a peak ratio of $128.8\times$ on a single instance. Greedy’s maximum runtime of 91.6 s ($n=493$, complete graph) approaches the 60 s time limit used by Chimani and

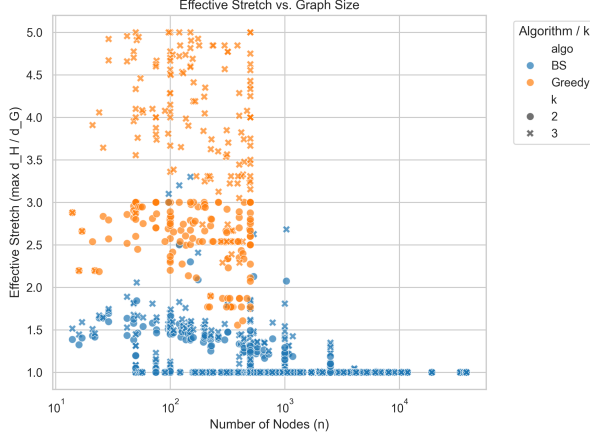


Figure 3: Effective stretch. Greedy consistently approaches α . BS under-utilizes available stretch, especially on sparse graphs.

Table 5: Wall-clock runtime (seconds) by family and algorithm.

Family	Algo	Mean	Max	Ratio
TSPLIB	Greedy	10.25	91.6	$33.7\times$
	BS	0.64	8.7	
Steinlib	Greedy	1.34	17.9	$\sim 6\times$
	BS	0.23	2.5	
Facebook	BS	1.5	—	—
Gnutella	BS	1.6	—	—

Stutzenstein [2022] and confirms that the $n \leq 500$ cap is binding. On Steinlib, Greedy averages 1.34 s and BS 0.23 s. BS scales continuously to $n=6,836$ (Steinlib) and $n=6,299$ (Gnutella) with runtimes under 2.6 s and 1.7 s respectively, consistent with the $O(km)$ complexity established in Chimani and Stutzenstein [2022]. The total ≈ 24 -hour experimental budget is dominated by Greedy runs on medium-to-large TSPLIB complete graphs and reflects the $O(n^4)$ effective cost in interpreted Python.

Figure 4 illustrates runtime scaling with graph size.

4.5 Peak Memory

Table 6 reports peak memory. Greedy’s footprint is small, at mean 1.55 MB on TSPLIB and 0.27 MB on Steinlib, reflecting its construction model where only the current partial spanner H and a priority queue are resident at any point. BS consumes approximately $5\times$ more memory on TSPLIB (mean

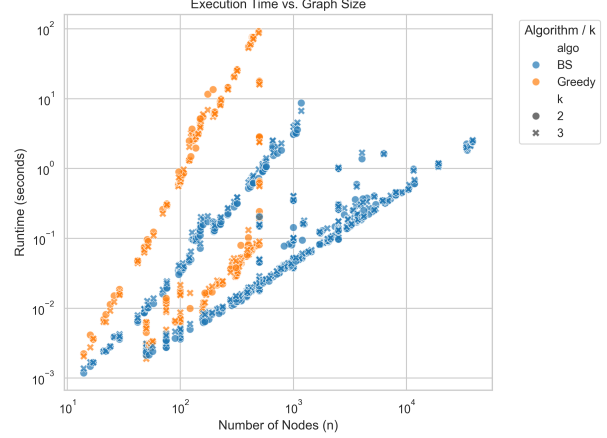


Figure 4: Runtime scaling. Greedy runtime grows steeply with n on dense graphs. BS runtime grows linearly in m .

Table 6: Peak memory consumption (MB) by family and algorithm.

Family	Algo	Mean	Max	Ratio
TSPLIB	Greedy	1.55	10.21	$\sim 5\times$
	BS	22.88	181.7	
Steinlib	Greedy	0.27	1.27	$\sim 3\times$
	BS	8.03	77.59	
Facebook	BS	48.84	57.15	—
Gnutella	BS	21.69	22.42	—

22.88 MB) and $3\times$ more on Steinlib (mean 8.03 MB). The elevated BS footprint arises from maintaining cluster membership tables, inter-cluster adjacency lists, and per-vertex nearest-cluster pointers across $k-1$ rounds simultaneously.

On the SNAP graphs, BS peaks at 57 MB (Facebook, $n=4,039$) and 22 MB (Gnutella, $n=6,299$). Memory scales super-linearly with n for BS on TSPLIB complete graphs, from 0.05 MB at $n=14$ rising to 181.7 MB at $n=1,173$, consistent with the $O(kn^{1+1/k})$ edge-count growth described in Chimani and Stutzenstein [2022].

Figure 5 shows peak memory as a function of graph size across both libraries.

5 Analysis and Discussion

5.1 Replication Fidelity

The primary finding of Chimani and Stutzenstein [2022], that Greedy produces the sparsest and lightest spanners while BS achieves the best runtime, is confirmed across all evaluated libraries. The di-

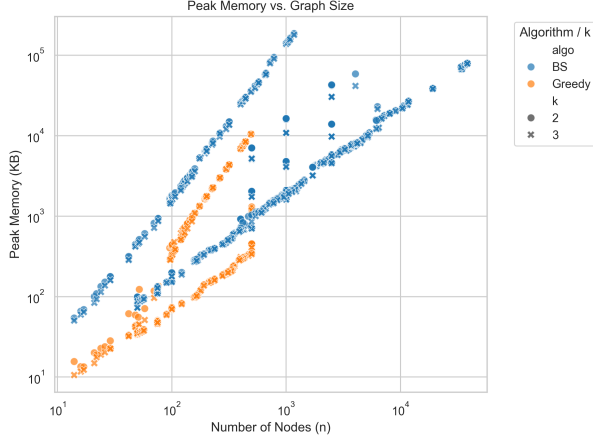


Figure 5: Peak memory consumption vs. graph size. BS requires 3–5 $\times$ more memory than Greedy on matched instances. Both scale super-linearly on dense graphs.

rectional conclusions hold in every configuration. Greedy sparseness is lower than BS sparseness on every matched instance. Greedy lightness is lower than BS lightness on every weighted instance. No instance reverses these rankings.

Quantitative magnitudes differ from Chimani and Stutzenstein [2022] primarily due to Python overhead (our Greedy cap excludes the largest instances) and the stratification of the Phase 2 Random Erdős–Rényi family evaluations from the primary real-world benchmark pipeline. The presence of TSPLIB (complete graphs, maximal density) and Steinlib (sparse real-world topologies) covers both ends of the density spectrum studied in the original paper.

5.2 Stretch–Sparseness Trade-off

The data reveals the trade-off identified by Chimani and Stutzenstein [2022]. Greedy’s high mean effective stretch (approaching α) and low sparseness confirm that it operates at the constraint boundary. It retains exactly the edges needed to satisfy the stretch requirement. BS’s low effective stretch and high sparseness reveal the opposite. Its distance-oblivious cluster sampling over-preserves distances, retaining edges that a stretch-aware algorithm would eliminate. Chimani and Stutzenstein [2022] attribute this directly to BS never performing any distance computation, which prevents it from exploiting the full stretch budget.

5.3 Implications of the Greedy Cap

The $n \leq 500$ cap excludes the largest instances from quality analysis. Since the $O(n^{1+1/k})$ sparseness guarantee established in Chimani and Stutzenstein [2022] improves relative to BS at larger scales, restricting to smaller instances understates the quality gap. The cap therefore makes Greedy’s advantage conservative, not inflated.

5.4 Baswana–Sen on Real-World Graphs

The SNAP graphs provide informative data points beyond the benchmark of Chimani and Stutzenstein [2022]. On the dense Facebook graph ($n=4,039$), BS achieves lightness 56–94 depending on α . Effective stretch stays below 1.05, confirming that BS’s conservative stretch behaviour extends to social network topology. On the sparse Gnutella graph ($n=6,299$), lightness is only 5.9 and stretch is 1.009, approaching isometric performance, consistent with the Steinlib pattern documented in Chimani and Stutzenstein [2022]. BS quality is therefore significantly better on sparse real-world topologies than on dense complete graphs.

5.5 Memory–Runtime Trade-off

The memory results reinforce the runtime findings of Chimani and Stutzenstein [2022]. BS pays a 3–5 $\times$ memory premium over Greedy to achieve its linear-time construction. On TSPLIB, BS’s maximum observed allocation of 181.7 MB (at $n=1,173$) is substantial but not prohibitive on modern hardware. However, the super-linear growth trend implies that very large graphs ($n \gg 10,000$) will require careful memory budgeting for BS. Greedy’s low footprint (≤ 10 MB across all evaluated instances) means memory is never a binding constraint for it within the $n \leq 500$ cap.

5.6 Lightness as the Critical Discriminator

Among the four quality metrics, lightness exhibits the most dramatic algorithmic differentiation (up to 804 $\times$ ratio on TSPLIB), confirming the conclusion of Chimani and Stutzenstein [2022] that “there is no alternative to ADDJS for applications requiring a lightness guarantee.” For applications in network design and routing where total edge weight matters, Greedy is unambiguously the correct choice at any tractable graph size.

6 Key Takeaways and Conclusions

Greedy dominates on quality. Across all evaluated instances and both libraries, Greedy produces spanners that are $2\text{--}12.5\times$ sparser and $5\text{--}90\times$ lighter than BS, reproducing the central experimental finding of Chimani and Stutzenstein [2022]. For weighted graphs requiring lightness bounds, Greedy is the only viable algorithm between the two.

BS dominates on scalability, at a memory cost. BS is $6\text{--}34\times$ faster per instance and scales to $n > 6,000$ where Greedy is computationally infeasible in Python. However, BS consumes $3\text{--}5\times$ more peak memory than Greedy on matched instances (up to 181.7 MB on TSPLIB at $n=1,173$), driven by cluster bookkeeping structures maintained across $k-1$ rounds. For unweighted or sparseness-tolerant applications on large graphs, BS remains the correct choice provided memory is available.

Stretch under-exploitation explains BS’s quality deficit. BS achieves mean effective stretch 1.06–1.51 against theoretical bounds of 3 or 5. The unused stretch budget directly translates to retained edges that a stretch-aware algorithm (Greedy) would have eliminated. This insight, first demonstrated empirically in Chimani and Stutzenstein [2022], is reproducibly confirmed in our Python implementation on a different hardware and language platform.

Python/NetworkX is a viable reproduction platform for $n \leq 500$. All qualitative and directional findings of Chimani and Stutzenstein [2022] are reproduced with fidelity. Quantitative runtimes are higher by $10\text{--}100\times$ due to interpreter overhead, but algorithmic behaviour is structurally identical. Reproducibility is achievable without compiled infrastructure for graphs up to several hundred vertices.

Future extensions. The most impactful additions are: (1) execution of the Baswana–Sen algorithm on the Erdős–Rényi synthetic graphs to complete the comparative baseline; (2) evaluation of higher stretch values ($\alpha \in \{4, 7\}$) to characterize the k -dependent trade-off; (3) memory profiling of BS at large scale; and (4) a Cython or PyPy port of Greedy to raise the practical size cap toward the $n \leq 1,200$ threshold reported in Chimani and Stutzenstein [2022].

References

Markus Chimani and Finn Stutzenstein. Spanner approximations in practice. In *Proceedings of the 30th Annual European Symposium on Algorithms (ESA 2022)*, volume 244 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 37:1–37:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022. doi: 10.4230/LIPIcs.ESA.2022.37.